Classification Assignment

1.)Identify your problem statement

Machine Learning
Supervised Learning
Classification

2.)Tell basic info about the dataset (Total number of rows, columns)

399 rows × 25 columns

3.) Mention the pre-processing method if you're doing any (like converting string to number – nominal data)

Yes, Converting Nominal to One Hot Encoding

4,5)

Develop a good model with good evaluation metric. You can use any machine learning algorithm; you can create many models. Finally, you have to come up with final model.

1.RandomForest Classifier

```
[18]: from sklearn.metrics import confusion_matrix
      cm=confusion_matrix(y_test,y_pred)

[19]: print(cm)

      [[50  1]
       [ 1 81]]

[20]: from sklearn.metrics import classification_report
      clf_report=classification_report(y_test,y_pred)

[21]: print(clf_report)

                   precision    recall  f1-score   support

              0       0.98      0.98      0.98        51
              1       0.99      0.99      0.99        82

       accuracy                           0.98       133
      macro avg       0.98      0.98      0.98       133
   weighted avg       0.98      0.98      0.98       133

[22]: print(X_train.shape)
      print(X_test.shape)

      (266, 27)
      (133, 27)
```

2. DecisionTree Classifier

[44]:
```
from sklearn.metrics import confusion_matrix
cm=confusion_matrix(y_test,y_pred)
```

[45]:
```
print(cm)
```

```
[[49  2]
 [ 8 74]]
```

[46]:
```
from sklearn.metrics import classification_report
clf_report=classification_report(y_test,y_pred)
```

[47]:
```
print(clf_report)
```

```
              precision    recall  f1-score   support

           0       0.86      0.96      0.91        51
           1       0.97      0.90      0.94        82

    accuracy                           0.92       133
   macro avg       0.92      0.93      0.92       133
weighted avg       0.93      0.92      0.93       133
```

[48]:
```
print(X_train.shape)
print(X_test.shape)
```

```
(266, 27)
(133, 27)
```

3.SVM Kernel Classification

[55]:
```
from sklearn.metrics import confusion_matrix
cm=confusion_matrix(y_test,y_pred)
```

[56]:
```
print(cm)
```

```
[[ 0 51]
 [ 0 82]]
```

[57]:
```
from sklearn.metrics import classification_report
clf_report=classification_report(y_test,y_pred)
```

```
C:\Anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:1731: UndefinedMetricWar
with no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
C:\Anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:1731: UndefinedMetricWar
with no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
C:\Anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:1731: UndefinedMetricWar
with no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
```

[58]:
```
print(clf_report)
```

```
              precision    recall  f1-score   support

           0       0.00      0.00      0.00        51
           1       0.62      1.00      0.76        82

    accuracy                           0.62       133
   macro avg       0.31      0.50      0.38       133
weighted avg       0.38      0.62      0.47       133
```

[59]:
```
print(X_train.shape)
print(X_test.shape)
```

```
(266, 27)
(133, 27)
```

4. Logistic Classification

[66]:
```
from sklearn.metrics import confusion_matrix
cm=confusion_matrix(y_test,y_pred)
```

[67]:
```
print(cm)
```

```
[[48  3]
 [ 5 77]]
```

[68]:
```
from sklearn.metrics import classification_report
clf_report=classification_report(y_test,y_pred)
```

[69]:
```
print(clf_report)
```

```
              precision    recall  f1-score   support

           0       0.91      0.94      0.92        51
           1       0.96      0.94      0.95        82

    accuracy                           0.94       133
   macro avg       0.93      0.94      0.94       133
weighted avg       0.94      0.94      0.94       133
```

[70]:
```
print(X_train.shape)
print(X_test.shape)
```

```
(266, 27)
(133, 27)
```

5.KNN

[77]:
```
from sklearn.metrics import confusion_matrix
cm=confusion_matrix(y_test,y_pred)
```

[78]:
```
print(cm)
```

```
[[40 11]
 [30 52]]
```

[79]:
```
from sklearn.metrics import classification_report
clf_report=classification_report(y_test,y_pred)
```

[80]:
```
print(clf_report)
```

```
              precision    recall  f1-score   support

           0       0.57      0.78      0.66        51
           1       0.83      0.63      0.72        82

    accuracy                           0.69       133
   macro avg       0.70      0.71      0.69       133
weighted avg       0.73      0.69      0.70       133
```

[81]:
```
print(X_train.shape)
print(X_test.shape)
```

```
(266, 27)
(133, 27)
```

6.Naive baye's

```
[88]: from sklearn.metrics import confusion_matrix
      cm=confusion_matrix(y_test,y_pred)
```

```
[89]: print(cm)
```

```
[[50  1]
 [23 59]]
```

```
[90]: from sklearn.metrics import classification_report
      clf_report=classification_report(y_test,y_pred)
```

```
[91]: print(clf_report)
```

```
              precision    recall  f1-score   support

           0       0.68      0.98      0.81        51
           1       0.98      0.72      0.83        82

    accuracy                           0.82       133
   macro avg       0.83      0.85      0.82       133
weighted avg       0.87      0.82      0.82       133
```

```
[92]: print(X_train.shape)
      print(X_test.shape)
```

```
(266, 27)
(133, 27)
```

6.)Final Justification

S.No	Model Name	Accuracy	Reason
1	RandomForest Classifier	**0.98**	Uses multiple decision trees (ensemble), reduces overfitting, gives highest accuracy
2	Logistic Regression	0.94	Works well for linear relationships, simple and efficient but less powerful for complex data
3	Decision Tree	0.92	Easy to understand, but tends to overfit compared to RandomForest
4	Naive Bayes	0.82	Fast and simple, but assumes features are independent (not always true)
5	K-Nearest Neighbors	0.69	Sensitive to noise and scaling, performance depends on K value
6	Support Vector Machine	0.62	Needs proper kernel tuning and scaling, otherwise gives low performance